

RUNBOOK — Claude Code plugins (the flygaca-family marketplace)

SECTION 06-operations-it

TYPE runbook

STATUS **active**

OWNER Founder

UPDATED 2026-08-26

Purpose. Install, use and maintain the family's Claude Code plugins so every session — in any of the three active repos, on any surface — starts with the same rules loaded instead of rediscovering them.

Scope. The marketplace lives in `ay2m/Office`. It carries four plugins: two in this repository, one in each product repository.

1. What exists

Plugin	Lives in	Covers
<code>office-docs</code>	<code>ay2m/Office</code> — <code>.claude/plugins/office-docs</code>	Front-matter, the print pipeline, the Arabic mirror, the entity-facts gate
<code>family-orchestrators</code>	<code>ay2m/Office</code> — <code>.claude/plugins/family-orchestrators</code>	Cross-repo workflows: full-sync, feature-ship, security sweep, compliance review
<code>flygaca-product</code>	<code>ay2m/FlyGACA</code> — <code>.claude/plugins/flygaca-product</code>	React surface, Express API, corpus pipelines, Postgres schema, the RAG flow
<code>captain-adel-service</code>	<code>ay2m/Captain-Adel</code> — <code>.claude/plugins/captain-adel-service</code>	Curriculum and the exam bank, provider operations, the corpus data layer, deployment

The catalog is `.claude-plugin/marketplace.json` at this repository's root. The two Office plugins are local relative-path entries; the two product plugins are `git-subdir` entries pointing at a path inside their own repository, so each product repo keeps ownership of its own plugin and nothing is copied between repos.

[!NOTE] The plugins **do not replace** the project-scoped agents and skills already in each repo's `.claude/` directory — `doc-smith` and `ar-mirror` here, `brain-retrieval` / `eval-warden` / `prompt-steward` / `site-chrome` in `ay2m/Captain-Adel`, `run-flygaca` in `ay2m/FlyGACA`. Those load automatically for a session inside that checkout. The plugins carry the knowledge a session **outside** the checkout would otherwise lack, plus the cross-repo workflows that belong to no single repo.

2. Install — Claude Code (terminal)

```
/plugin marketplace add ay2m/Office
/plugin install family-orchestrators@flygaca-family
```

Repeat `/plugin install <name>@flygaca-family` for each plugin you want. The install prompts for a **scope**:

Scope	Effect
User	You, in every project
Project	Everyone on this repository (writes to <code>.claude/settings.json</code>)
Local	You, in this repository only

If the install summary says `Run /reload-plugins to activate.`, run it.

Non-interactive equivalent, for scripting or a fresh machine:

```
claude plugin marketplace add ay2m/Office
claude plugin install family-orchestrators@flygaca-family --scope project
```

`ay2m/Office` is **private**, so adding the marketplace requires a git identity that can read it. That is the intended access control — the marketplace is not public, and it should not become public: it is hosted in the documents repository.

3. Install — cloud sessions and the desktop app

- **Claude Code on the web / cloud sessions.** `/plugin` is an interactive terminal panel and is not available there. Declare the plugin under `enabledPlugins` in `.claude/settings.json` instead.
- **Claude desktop app.** Use its plugin browser rather than the slash command.
- **Cursor, VS Code, JetBrains.** Plugins are a **Claude Code** feature, not an editor feature. You get them by running Claude Code inside the editor — its integrated terminal or the IDE extension — and they behave exactly as they do in a standalone terminal. Cursor's own rules format is a separate system; do not expect these plugins to configure it, and do not duplicate their content into `.cursor/` where it would drift.

4. Team onboarding (the repo does the work)

All three repos now register the marketplace in their project settings, so a teammate who trusts the folder gets the catalog without being told to add it:

```
{
  "extraKnownMarketplaces": {
    "flygaca-family": {
      "source": { "source": "github", "repo": "ay2m/Office" }
    }
  }
}
```

Registering a marketplace **does not install anything**. A plugin whose source is external — which is every `git-subdir` entry here — still has to be installed explicitly; Claude Code reports it as not installed and prints the `claude plugin install` command to run. That is deliberate: nobody's session starts executing plugin content because they cloned a repo.

A new engineer's first day, in order:

1. Clone the repo and trust the folder.
2. `/plugin install <the plugin for this repo>@flygaca-family`.
3. `/plugin` → **Installed** tab to confirm what loaded, and the **Errors** tab if a command or agent is missing.
4. Read the repo's `CLAUDE.md`. It remains authoritative — the plugin is a distribution mechanism for its rules, not a replacement for them.

5. Maintaining a plugin

Plugin content is plain Markdown with YAML front-matter, so editing one is editing a file in the repo it belongs to.

- **Agents** live in `agents/*.md`: `name` matches the filename, `description` is written in task language ("use proactively when..."), and `model` is omitted so each agent inherits the session's.
- **Commands** live in `commands/*.md` and are invoked namespaced — `/flygaca-product:verify`. (Claude Code now suggests `skills/<name>/SKILL.md` for new plugins; these stay flat commands because they are user-invoked procedures that use `argument-hint` and `allowed-tools`. Revisit if that changes.)
- **Bump version** in `.claude-plugin/plugin.json` when behaviour changes, and the matching `version` in this repository's `marketplace.json` entry.
- **Validate before pushing:** `claude plugin validate <path-to-plugin>` runs the same check the submission pipeline runs. Test locally without installing: `claude --plugin-dir ./claude/plugins/<name>`, then `/reload-plugins` after each edit.
- **Never put** `agents/`, `commands/`, `skills/` or `hooks/` **inside** `.claude-plugin/`. Only `plugin.json` goes there; everything else sits at the plugin root. This is the most common structural mistake.

Where content changes. The product plugins are owned by their repos: fix `flygaca-product` in `ay2m/FlyGACA` and `captain-adel-service` in `ay2m/Captain-Adel`. Only the catalog entry lives here. A plugin whose agent contradicts its repo's `CLAUDE.md` is a bug in the plugin, not in `CLAUDE.md`.

Doc-gate note

`.claude/` is in `SKIP_DIRS` for `check.mjs` / `build.mjs` / `build-html.mjs`, so plugin Markdown needs **no** front-matter and **no** `_print/` PDF. That skip is root-anchored — the plugins must stay under `.claude/plugins/`. This runbook, of course, is a normal document and follows the normal rules.

6. Security

Plugins execute with your privileges and can carry hooks and MCP servers. Ours carry neither today — they are agents and commands, i.e. instructions — but the rule stands: **only add marketplaces and install plugins you trust**. Review a diff to a plugin the way you would review a change to CI.

Two family-specific cautions:

- The marketplace is hosted in the documents repository, which holds real legal, financial, HR and investor material. Plugin content must never quote it.
- If a plugin ever gains a hook, an MCP server or a `bin/` executable, that is a security review, not a docs change — route it per [01-governance/SECURITY.md](#).

7. Troubleshooting

Symptom	Cause / fix
<code>/plugin</code> unknown command	Old CLI. Update Claude Code, restart the terminal.
Marketplace not found	Not added in this scope: <code>/plugin marketplace add ay2m/Office</code> .
Plugin listed as not installed after cloning	Expected for external sources — run the <code>claude plugin install</code> command it prints.
Commands missing after an edit	<code>/reload-plugins</code> (<code>--force</code> if it warns about the prompt cache).
Still missing	<code>rm -rf ~/.claude/plugins/cache</code> , restart, reinstall.
Marketplace add fails on a private repo	The git identity can't read <code>ay2m/Office</code> . Fix access; do not make the repo public.

8. Related

- [_INDEX.md](#) — the master index
- [06-operations-it/flygaca-claude-briefing.md](#) — the standalone briefing for chat sessions with **no** repo access; the plugins are the repo-access answer to the same problem
- [01-governance/SECURITY.md](#) — where a security concern goes
- [contracts/flygaca-family.json](#) — the cross-repo family contract the orchestrators audit